

 **c0a25046bf** 基本コード

492c24d · last week 

31 lines (23 loc) · 1.23 KB

Preview

Code

Blame

 

Raw

 

実行環境の必要条件

- ## ゲームの概要

- # ゲームの遊び方

- ## ゲームの実装

共通基本機能

- 矢印でジャンプの方向決めてスペースの押す長さでジャンプ力を決める
- 壁に当たると跳ね返る

分擔追加機能

- 1_グラフィック（画像）変更[],2_当たり判定の厳密化（頭ぶつけ・すり抜け防止）[],3_ステージギミック追加[],4_真上ジャンプ機能[],5_左上のUI（高さ・情報の表示）[]

ToDo

メモ

- クラス内の変数は、すべて、「get_変数名」という名前のメソッドを介してアクセスするように設計してある
- すべてのクラスに関係する関数は、クラスの外で定義してある
`os.chdir(os.path.dirname(os.path.abspath(file)))`

 **c0a251904e** 左上のUIデザイン変更 f022b89 · 8 minutes ago 

```

Code Blame
import pygame
import sys
import os
import random

# カレントディレクトリの固定
os.chdir(os.path.dirname(os.path.abspath(__file__)))

WIDTH, HEIGHT = 600, 800
FPS = 60
GRAVITY = 0.6

BLACK = (20, 20, 25)
BLUE = (100, 150, 255)
GREEN = (120, 120, 120)
RED = (255, 100, 100)
WHITE = (255, 255, 255)
GOLD = (255, 215, 0) # ゴールブロック用の金色

DARK_PANEL = (35, 35, 45)
PANEL_BORDER = (90, 90, 110)
LIGHT_BLUE = (120, 200, 255) # 左上UI
ORANGE = (255, 170, 80)

# =====
# 1. データ構造 (クラス) の定義
# =====

class Player:
    def __init__(self):
        self._image = pygame.Surface((30, 40))
        self._image.fill(BLUE)
        self._rect = self._image.get_rect(center=(WIDTH // 2, HEIGHT // 2))

        self._vel_x = 0
        self._vel_y = 0
        self._on_ground = False

        self._is_charging = False
        self._charge_power = 0
        self._max_charge = 20.0

```



```

43         self._direction = 1
44
45         self._is_clear = False # ゴールしたかどうかのフラグ
46
47     # --- ゲッター ---
48     def get_image(self): return self._image
49     def get_rect(self): return self._rect
50     def get_vel_x(self): return self._vel_x
51     def get_vel_y(self): return self._vel_y
52     def get_on_ground(self): return self._on_ground
53     def get_is_charging(self): return self._is_charging
54     def get_charge_power(self): return self._charge_power
55     def get_max_charge(self): return self._max_charge
56     def get_direction(self): return self._direction
57     def get_is_clear(self): return self._is_clear
58
59     # --- セッター ---
60     def set_vel_x(self, value): self._vel_x = value
61     def set_vel_y(self, value): self._vel_y = value
62     def set_on_ground(self, value): self._on_ground = value
63     def set_is_charging(self, value): self._is_charging = value
64     def set_charge_power(self, value): self._charge_power = value
65     def set_direction(self, value): self._direction = value
66     def set_is_clear(self, value): self._is_clear = value
67
68
69     class Platform:
70         # 足場ごとに色を変えられるように引数追加
71         def __init__(self, x, y, w, h, color):
72             self._image = pygame.Surface((w, h))
73             self._image.fill(color)
74             self._rect = self._image.get_rect(topleft=(x, y))
75
76         def get_image(self): return self._image
77         def get_rect(self): return self._rect
78
79
80         # =====
81         # 2. クラスを操作する関数
82         # =====
83
84     def update_player(player, keys, platforms, goal_block):
85         """プレイヤーの物理挙動と状態を更新する関数"""
86
87         # ゴール済みの場合は入力を受け付けない（重力で落ちるだけ）
88         if not player.get_is_clear():
89             # 1. 地面にいる時の処理
90             if player.get_on_ground():
91                 player.set_vel_x(0)
92
93                 if keys[pygame.K_LEFT] or keys[pygame.K_a]:
94                     player.set_direction(-1)
95                 if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
96                     player.set_direction(1)
97
98                 if keys[pygame.K_SPACE]:
99                     player.set_is_charging(True)
100                     current_power = player.get_charge_power()
101                     if current_power < player.get_max_charge():

```

```

102         player.set_charge_power(current_power + 1)
103     else:
104         if player.get_is_charging():
105             power = player.get_charge_power()
106             direction = player.get_direction()
107
108             player.set_vel_y(-power * 0.9 - 5)
109             player.set_vel_x(direction * (power * 0.9 - 5))
110
111             player.set_is_charging(False)
112             player.set_charge_power(0)
113             player.set_on_ground(False)
114
115     # 2. 重力の適用
116     if not player.get_on_ground():
117         player.set_vel_y(player.get_vel_y() + GRAVITY)
118
119     rect = player.get_rect()
120
121     # 座標の更新 (x軸) と壁の跳ね返り
122     rect.x += player.get_vel_x()
123     if rect.left < 0:
124         rect.left = 0
125         player.set_vel_x(player.get_vel_x() * -0.5)
126         player.set_direction(1)
127     if rect.right > WIDTH:
128         rect.right = WIDTH
129         player.set_vel_x(player.get_vel_x() * -0.5)
130         player.set_direction(-1)
131
132     # 座標の更新 (y軸)
133     rect.y += int(player.get_vel_y())
134
135     # --- 3. ゴール (天井) との当たり判定 ---
136     goal_rect = goal_block.get_rect()
137     if rect.colliderect(goal_rect):
138         # 上昇中に天井の底にぶつかったらゴール
139         if player.get_vel_y() < 0:
140             rect.top = goal_rect.bottom
141             player.set_vel_y(0)
142             player.set_is_clear(True)
143
144     # --- 4. 足場との当たり判定 ---
145     player.set_on_ground(False)
146     if player.get_vel_y() > 0:
147         for plat in platforms:
148             plat_rect = plat.get_rect()
149             if rect.colliderect(plat_rect):
150                 if rect.bottom <= plat_rect.top + player.get_vel_y():
151                     rect.bottom = plat_rect.top
152                     player.set_vel_y(0)
153                     player.set_vel_x(0)
154                     player.set_on_ground(True)
155                     break
156
157     def draw_ui(screen, font, player, current_floor, total_floors):
158
159         # UIのサイズ
160         panel_x = 10

```

```

161     panel_y = 10
162     panel_w = 300
163     panel_h = 120
164
165     # 背景のパネル
166     panel_rect = pygame.Rect(panel_x, panel_y, panel_w, panel_h)
167     pygame.draw.rect(screen, DARK_PANEL, panel_rect, border)
168     pygame.draw.rect(screen, PANEL_BORDER, panel_rect, 2, border)
169     # タイトル
170     title_text = font.render("STATUS", True, GOLD)
171     screen.blit(title_text, (panel_x + 15, panel_y + 10))
172
173     # 階層
174     floor_text = font.render(f"Floor {current_floor} / {total_floors}")
175     screen.blit(floor_text, (panel_x + 15, panel_y + 42))
176
177     # 高さ
178     height_text = font.render(f"Height: {current_height // 10} m")
179     screen.blit(height_text, (panel_x + 15, panel_y + 70))
180
181     # 最高到達点
182     max_text = font.render(f"Best: {max_height // 10} m", True, GOLD)
183     screen.blit(max_text, (panel_x + 180, panel_y + 70))
184
185
186
187
188     # =====
189     # 3. メインループ
190     # =====
191
192     def main():
193         pygame.init()
194         screen = pygame.display.set_mode((WIDTH, HEIGHT))
195         pygame.display.set_caption("Jump King - 10 Floors to Go!")
196         clock = pygame.time.Clock()
197         font = pygame.font.SysFont(None, 36)
198         large_font = pygame.font.SysFont(None, 72)
199
200         player = Player()
201
202         # --- 10層分のマップ生成 ---
203         platforms = []
204         platforms.append(Platform(0, HEIGHT - 40, WIDTH, 40, GREY))
205
206         TOTAL_FLOORS = 10
207         # ゴール（天井）のY座標：スタート床から 10層分(800px * 10)
208         goal_y = (HEIGHT - 40) - (HEIGHT * TOTAL_FLOORS)
209
210         # 金色の天井ブロックを生成
211         goal_block = Platform(0, goal_y, WIDTH, 40, GOLD)
212
213         # ゴールに到達するまで足場を生成し続ける
214         platform_y = HEIGHT - 150
215         while platform_y > goal_y + 100:
216             w = random.randint(60, 130)
217             x = random.randint(0, WIDTH - w)
218             platforms.append(Platform(x, platform_y, w, 20, GREY))
219             gap_y = random.randint(80, 130)

```

```
220         platform_y -= gap_y
221
222     camera_y = 0
223     max_height = 0
224
225     running = True
226     while running:
227         for event in pygame.event.get():
228             if event.type == pygame.QUIT:
229                 running = False
230
231         keys = pygame.key.get_pressed()
232
233         # 関数呼び出し (ゴールブロックも渡して判定させる)
234         update_player(player, keys, platforms, goal_block)
235
236         p_rect = player.get_rect()
237
238         # --- カメラ処理 ---
239         SCROLL_TOP_MARGIN = 200
240         if p_rect.y - camera_y < SCROLL_TOP_MARGIN:
241             camera_y = p_rect.y - SCROLL_TOP_MARGIN
242
243         SCROLL_BOTTOM_MARGIN = 150
244         camera_easing_down = 0.08
245         if p_rect.bottom - camera_y > HEIGHT - SCROLL_BOTTOM_MARGIN:
246             target_camera_y = p_rect.bottom - (HEIGHT - SCROLL_BOTTOM_MARGIN)
247             camera_y += (target_camera_y - camera_y) * camera_easing_down
248
249         if camera_y > 0:
250             camera_y = 0
251
252         # --- スコア・階層計算 ---
253         current_height = (HEIGHT - 40) - p_rect.bottom
254         if current_height > max_height:
255             max_height = current_height
256
257         # 今いる階層 (800px を 1階 とする)
258         current_floor = min((current_height // HEIGHT) + 1,
259                             platforms[current_floor].floor)
260
261         # --- 描画処理 ---
262         screen.fill(BLACK)
263
264         # ゴールの描画
265         goal_rect = goal_block.get_rect()
266         goal_draw_y = goal_rect.y - camera_y
267         if -100 < goal_draw_y < HEIGHT + 100:
268             screen.blit(goal_block.get_image(), (goal_rect.x, goal_draw_y))
269
270         # 足場の描画
271         for plat in platforms:
272             plat_rect = plat.get_rect()
273             draw_y = plat_rect.y - camera_y
274             if -50 < draw_y < HEIGHT + 50:
275                 screen.blit(plat.get_image(), (plat_rect.x, draw_y))
276
277         # プレイヤーの描画
278         player_draw_y = p_rect.y - camera_y
279         screen.blit(player.get_image(), (p_rect.x, player_draw_y))
```

```
279
280     # チャージゲージと方向指示器
281     if player.get_is_charging() and not player.get_is_c
282         gauge_width = (player.get_charge_power() / play
283         pygame.draw.rect(screen, RED, (p_rect.centerx -
284
285     arrow_x = p_rect.right + 5 if player.get_direction(
286     pygame.draw.rect(screen, WHITE, (arrow_x, player_dr
287
288
289     # 情報表示 (UI)
290     draw_ui(screen, font, player, current_floor, TOTAL_
291     # ゴールした時の演出
292     if player.get_is_clear():
293         clear_text = large_font.render("CLEAR!!", True,
294         text_rect = clear_text.get_rect(center=(WIDTH /
295         screen.blit(clear_text, text_rect)
296
297         sub_text = font.render("CONGRATULATIONS!", True
298         sub_rect = sub_text.get_rect(center=(WIDTH // 2
299         screen.blit(sub_text, sub_rect)
300
301     pygame.display.flip()
302     clock.tick(FPS)
303
304     pygame.quit()
305     sys.exit()
306
307 if __name__ == "__main__":
308     main()
```


157	157	+ def draw_ui(screen, font, player, current_floor, total_floors, current_height, max_height):
158	158	+
159	159	+ # UIのサイズ
160	160	+ panel_x = 10
161	161	+ panel_y = 10
162	162	+ panel_w = 300
163	163	+ panel_h = 120
164	164	+
165	165	+ # 背景のパネル
166	166	+ panel_rect = pygame.Rect(panel_x, panel_y, panel_w, panel_h)
167	167	+ pygame.draw.rect(screen, DARK_PANEL, panel_rect, border_radius=12)
168	168	+ pygame.draw.rect(screen, PANEL_BORDER, panel_rect, 2, border_radius=12)
169	169	+ # タイトル
170	170	+ title_text = font.render("STATUS", True, GOLD)
171	171	+ screen.blit(title_text, (panel_x + 15, panel_y + 10))
172	172	+
173	173	+ # 階層
174	174	+ floor_text = font.render(f"Floor {current_floor} / {total_floors}", True, WHITE)
175	175	+ screen.blit(floor_text, (panel_x + 15, panel_y + 42))
176	176	+
177	177	+ # 高さ
178	178	+ height_text = font.render(f"Height: {current_height // 10} m", True, LIGHT_BLUE)
179	179	+ screen.blit(height_text, (panel_x + 15, panel_y + 70))
180	180	+
181	181	+ # 最高到達点
182	182	+ max_text = font.render(f"Best: {max_height // 10} m", True, ORANGE)
183	183	+ screen.blit(max_text, (panel_x + 180, panel_y + 70))
184	184	+
185	185	+
186	186	+
151	187	
152	188	# =====
153	189	# 3. メインループ
249	285	arrow_x = p_rect.right + 5 if player.get_direction() == 1 else p_rect.left - 10
250	286	pygame.draw.rect(screen, WHITE, (arrow_x, player_draw_y + 15, 5, 5))
251	287	
252		- # 情報表示 (UI)
253		- ui_text = font.render(f"Floor: {current_floor} / {TOTAL_FLOORS} Max: {max_height // 10}m", True, WHITE)
254		- screen.blit(ui_text, (10, 10))
255	288	
	289	+ # 情報表示 (UI)
	290	+ draw_ui(screen, font, player, current_floor, TOTAL_FLOORS, current_height, max_height)
256	291	# ゴールした時の演出
257	292	if player.get_is_clear():
258	293	clear_text = large_font.render("CLEAR!!", True, GOLD)